

Migration Approach Comparison & WeighApp Dependency Analysis

Two working discussions on the AIS-to-ROS2 port strategy for LpsSaJobMgrApp / LpsSaWeighApp

Part I — Full rewrite vs. communication-only: an unbiased comparison

Part II — WeighApp's actual file/folder dependency footprint, and what each approach means for it

Part I — Approach comparison

Two approaches came up for porting these apps to ROS2: **Approach 1** — rewrite the app's logic natively as ROS2 code (feature-by-feature or whole-app), eventually retiring the AIS binary entirely. **Approach 2** — swap only the communication layer to ROS2/DDS, leaving the core logic and processing exactly as it is today. Below is an unbiased weighing of both, not just a restatement of a preference.

APPROACH 1

Full rewrite, native ROS2

REAL ADVANTAGES

- Clean end state — no permanent shim/adaptor layer, no two parallel type systems (SCS struct + ROS2 msg) to keep in sync by hand forever.
- A natural point to fix known issues already found in this codebase: the multi-task command/display SEA asymmetry (commands aren't gated where the UI display is), the inverted diagnostic condition in `LpsSaProcess.cpp`'s tilt-HE-voltage check, and `WeightConditionsMet()` being removed with no obvious replacement. A rewrite forces a decision on each; a mechanical port carries them forward faithfully either way, bugs and all.
- Idiomatic code going forward — nothing to explain to a future engineer beyond "this is a ROS2 node."
- No risk of the shim layer itself having subtle bugs (e.g. not correctly replicating SCS's drain-queue `get()` semantics).

REAL COSTS

- Every feature has to be independently re-verified, not just the I/O boundary. Mapping JobMgr's 21 features and their interactions took significant effort on its own; reimplementing them correctly is riskier still — the subtle stuff (negative findings like "no SEA check gates ZERO/CLEAR/MINUS_ONE," exact ordering of `store_request_status` vs `loadUpdate.store`) is exactly what gets silently lost in a rewrite.
- **Doesn't scale to "eventually all 16 apps."** A bespoke reimplementation-and-reverification effort per app, 16 times,

APPROACH 2

Communication-only, logic stays

REAL ADVANTAGES

- Verified concretely on JobMgr: **7 of 9 logic files need zero changes**, 12 SCS touch-points total out of ~3,662 lines. The risk surface is genuinely small.
- Scales — the pattern (find the file(s) that touch SCS, wrap them, leave everything else) is mechanical and repeatable per app.
- A working precedent for this exact pattern already exists in this repo: `cpm_scs_bridge`'s prototype is built as an AIS `task::Task` with an embedded ROS2 node spun each `executive()` tick — meaning the process doesn't even have to leave the AIS scheduling/watchdog/priority model to do this.
- Testing is bounded — verify the I/O boundary and do an end-to-end regression pass, not re-derive correctness of logic that wasn't touched.

REAL COSTS

- Permanent maintenance burden. The shim layer and the "keep ROS2 msg field names identical to SCS struct field names" convention are forever obligations — every future field change needs updating in two places, by discipline, not by the compiler catching it.
- Doesn't reach a "pure ROS2" end state — the internals still think in SCS's request/response/ polling idioms. A permanent hybrid unless a rewrite happens later anyway.
- Known quirks ship forward unexamined — nothing forces a decision on whether the SEA asymmetry or the inverted diagnostic

especially for apps with hardware/real-time/regulatory attachments, is a fundamentally different scale of project than a repeatable transport-swap pattern.

- Loses field-proven status — the current binaries have run on real machines for years; every rewritten line starts back at zero trust.
- For WeighApp specifically, most plausibly triggers Legal-For-Trade re-certification, since the actual measurement computation would be new code, not just newly-transported data.

condition are bugs; they just keep running exactly as they do today.

- If the AIS/OEL toolchain itself hits end-of-life, this doesn't insulate from that — the shell rewrite is still needed eventually, just later, with the transport question already answered.

The actual read

For this project, at this scale, Approach 2 wins — but the reasoning matters more than the conclusion. The deciding factor isn't "logic-preservation is inherently better." It's that the stated scope is **16 apps, several with hardware and regulatory attachments**, and only one of the two approaches has a cost curve that doesn't explode at that scale. A full rewrite's cost is dominated by re-verification effort per app; a transport swap's cost is dominated by how much of each app already touches SCS directly — which, per JobMgr, is a small fraction. That asymmetry is the whole argument, and it holds regardless of which app comes next.

Where I'd reconsider: if a future app's file-by-file SCS-touch count comes back large, or its "logic" files turn out to be more entangled with SCS structures than JobMgr's were, the cost/benefit shifts back toward Approach 1 for *that specific app* — this isn't a rule to apply uniformly, it's a rule to re-measure per app the way it was just measured for JobMgr.

Where to be honest about the trade-off: Approach 2 as a permanent architecture means the known quirks found so far stay in the system indefinitely, unexamined. If any of those turn out to matter (the SEA asymmetry in particular seems worth a second look — commands not gated where the display is), that's worth a deliberate decision now, separate from the transport question, rather than something a rewrite would have surfaced automatically.

Part II — WeighApp's actual dependency footprint

"Rewrite WeighApp" doesn't mean touching one app folder. WeighApp's build directly compiles source files from several other top-level projects straight into its own binary, and some of those are shared with apps that aren't in scope for this migration at all.

Compiled straight into WeighApp's binary

Its `SConscript` globs `.c` files from these folders and builds them directly into `LpsSaWeighApp` — not linked as a library, compiled in.

Folder	Path	Status here	What it is
<code>lps_weighing</code>	<code>/lps_weighing/src/*.c</code>	Missing (uninitialized submodule)	The actual weighing computation — <code>LpsInit()</code> / <code>LpsUpdt()</code> , referred to throughout this analysis as "the compiled weighing library."
<code>lps_cal</code>	<code>/lps_cal/src/*.c</code> + <code>imu/*.c</code>	Present	Calibration library — <code>LpsCalInit()</code> / <code>LpsCalUpdate()</code> . Verified: zero AIS/SCS includes in its own source — plain C, same pattern as <code>lps_pass_tracker</code> .
<code>lps_common</code>	<code>/lps_common/src/*.c</code> (3 specific files)	Missing	Shared math utilities — lookup tables, weight conversion, filters.
<code>tipoff_assist</code>	<code>/tipoff_assist/toa_wl00*</code>	Missing	The actual Tip-Off Assist model — Simulink-autocoded, given the <code>_model_if.c</code> naming.

Header-only, external platform SDK

Not vendored anywhere in this workspace — same situation as `scsIOContainer` established at the very start of this whole investigation. This isn't source you could read, let alone rewrite, from here:

`ais/*` (the AIS task framework itself) · `autonomyConditions/*` · `commTask/*` · `mlComms/*` · `scsIOContainer/*` · `fileio/*` · `hub/*`

Include-only, present in-repo (or not)

`lps_app_common/include` (present) · `boost` (third-party) · `TES-common-ais/.../chrono` (missing) · `libsl` — Simulink runtime headers (missing)

Who else depends on the same shared libraries

This is the part that matters most: several of WeighApp's dependencies are genuinely shared with apps outside this migration's scope.

Shared library	Also used by
<code>lps_common</code>	<code>LpsSaJobMgrApp</code> , <code>LpsSaListMgrApp</code> , <code>LpsSaTotalsApp</code>
<code>lps_app_common</code>	<code>LpsSaJobMgrApp</code> , <code>DemoApp</code>
<code>lps_cal</code>	<code>aisXcpServer</code>
<code>libsl</code>	<code>CycleSeg</code> , <code>DemoApp</code>
<code>lps_weighing</code> , <code>tipoff_assist</code>	WeighApp only

What Approach 1 (full rewrite) would concretely mean here

1. `lps_weighing` and `tipoff_assist` aren't even visible right now. Before any rewrite of the core weighing math could start, the actual submodules would need to be pulled from wherever they live — they're referenced in `.gitmodules` but not populated in this checkout. That's reimplementing physics that hasn't been read yet.
2. `lps_common` is used by three other apps not in scope for this migration (`ListMgrApp`, `TotalsApp`, plus `JobMgr` which is in scope). If "full rewrite" means reimplementing its math natively in the new ROS2 code rather than keeping the same compiled library, that produces two implementations of the same weight-conversion/filter logic — one the old AIS apps still use, one the new ROS2 code uses — with real risk of silent drift between them on any future correction.
3. `lps_app_common` — same problem, shared with `JobMgr` and `DemoApp`.
4. `lps_cal` — shared with `aisXcpServer`, the calibration/ diagnostic tool. If WeighApp's rewrite changes how calibration data is structured, the diagnostic tool reading it (still AIS-side) may silently stop understanding it.
5. The **header-only externals** (`ais/*` , `scsIOContainer/*` , `mlComms/*`) aren't available to read at all — a full rewrite of anything depending on them means guessing at behavior that can't be inspected.

Approach 1 for WeighApp isn't "rewrite one app" — it's "decide the fate of 4 shared library projects, two of which aren't currently visible, at least two of which are load-bearing for apps that aren't being migrated."

What Approach 2 (communication-only) means for the same map

Every row in the dependency table stays exactly as it is. `lps_weighing` , `lps_cal` , `lps_common` , `lps_app_common` , `tipoff_assist` keep getting compiled into WeighApp's binary exactly the way they do today — the only difference is what happens to the struct after `LpsSaWeighingScsTx()` builds it: publish to SCS, or publish to a ROS2 topic. The other apps sharing `lps_common` / `lps_app_common` / `lps_cal` never

know anything changed, because nothing about those libraries did. `lps_weighing` and `tipoff_assist` don't even need to be populated to reason about this correctly, since their internals are never read — only whatever comes out of `LpsUpdt()` gets relayed to a different transport.

Bottom line: the WeighApp-specific dig makes the general case from Part I concrete. The "one feature touches many files, some shared with other apps" risk is real, verified, and specifically hits `lps_common` and `lps_app_common` (shared with JobMgr, ListMgrApp, TotalsApp, DemoApp) and `lps_cal` (shared with aisXcpServer) — all apps outside the current migration scope. Approach 1 forces a choice between forking that shared code or coordinating a multi-app migration around it. Approach 2 sidesteps the question entirely because it never touches those files.

Source: D:\ROS2_Service_Discovery\cpm\ssp-CPM_Loader-ais\ssp-CPM_Loader-ais — verified via direct SConscript inspection, submodule population checks, and cross-app SConscript search for shared library references.